

CADENAS

Ya hemos visto que se pueden crear con comillas simples (" , sin interpretación) o comillas dobles ("" , interpretan el contenido y las secuencias de escape \n, \t, \\$, {, ... - magic quotes)

```
<?php
"Me llamo $nombre"
"Son 30 {$moneda}s"
?>
```

Se acceden a los caracteres como si fuera un array.

```
<?php
$cadena = "Yo soy Batman" ;
$ygriega = $cadena[0];
?>
```

OPERACIONES BÁSICAS

Todas las funciones se pueden consultar en <https://www.php.net/manual/es/ref.strings.php>

Las más importantes son:

- 1.*strlen*: obtiene la longitud de una cadena y devuelve un número entero
- 2.*substr*: devuelve una subcadena de la cadena original
- 3.*str_replace*: reemplaza caracteres en una cadena
- 4.*strtolower* y *strtoupper*: Transforman una cadena de caracteres en la misma cadena en minúsculas o mayúsculas respectivamente.

```
<?php
$cadena = "El caballero oscuro";
$tam = strlen($cadena);
echo "La longitud de '$cadena' es: $tam <br />";

$oscuro = substr($cadena, 13); // desde 13 al final
$caba = substr($cadena, 3, 4); // desde 3, 4 letras
$katman = str_replace("c", "k", $cadena);
echo "$oscuro $caba ahora es $katman";

echo "Grande ".strtoupper($cadena);
?>
```

Si queremos limpiar cadenas, tenemos la función:

- 1.*trim*: elimina los espacios al principio y al final

```
<?php
$cadena = " Programando en PHP ";
$limpia = trim($cadena); // "Programando en PHP"
```

?>

Comparando y Buscando

La comparación de cadenas puede ser con conversión de tipos mediante `==` o estricta con `===`. También funcionan los operadores `<` y `>` si ambas son cadenas. Al comparar cadenas con valores numéricos podemos utilizar:

1.*strcmp*: 0 iguales, `<0` si "a < b" o `>0` si "a > b"

2.*strcasecmp*: las pasa a minúsculas y compara

```
<?php
$frase1 = "Alfa";
$frase2 = "Alfa";
$frase3 = "Beta";
$frase4 = "Alfa5";
$frase5 = "Alfa6";

var_dump( $frase1 == $frase2 ); // true
var_dump( $frase1 === $frase2 ); // true
var_dump( strcmp($frase1, $frase2) ); // 0
var_dump( $frase2 < $frase3 ); // true
var_dump( strcmp($frase2, $frase3) ); // -1
var_dump( strcmp($frase4, $frase5) ); // 1
```

?>

Si lo que queremos es buscar dentro de una cadena, tenemos:

1.*strpos* / *strrpos*: busca en una cadena y devuelve la posición de la primera/última ocurrencia.

2.*strstr* / *strchr* (*alias*): busca una cadena y devuelve la subcadena a partir de donde la ha encontrado

```
<?php
$frase = "Quien busca encuentra, eso dicen, a veces";
$pos1 = strpos($frase, ","); // encuentra la primera coma
$pos2 = strrpos($frase, ","); // encuentra la última coma
$trasComa = strstr($frase, ","); // ", eso dicen, a veces"
?>
```

Trabajando con subcadenas

Si queremos romper las cadenas en trozos, tenemos:

1.*explode*: convierte en array la cadena mediante un separador.

2.*implode* / *join*: pasa un array a cadena con un separador

```
<?php
$frase = "un superpoder, requiere una gran responsabilidad, dijo el tio de Spiderman";
$partes = explode(",", $frase);
```

```
$ciudades = ["Valencia", "Torrent", "Gandía"];  
$cadenaCiudades = implode(">", $ciudades);
```

Finalmente, otras operaciones que podemos realizar para trabajar con subcadenas son:

1.*substr_count*: número de veces que aparece la subcadena dentro de la cadena

2.*substr_replace*: reemplaza parte de la cadena a partir de su posición, y opcionalmente, longitud

```
<?php  
$batman = "Bruce Wayne es Batman";  
$empresa = substr($batman, 6, 5); // Wayne  
$bes = substr_count($batman, "B"); // 2
```

```
?>
```

También disponemos de una serie de funciones que facilitan las codificaciones desde y hacia HTML:

1.*htmlentities*: convierte a entidades HTML, por ejemplo, á por á, ñ por ñ, < por <, etc..

2.*htmlspecialchars*: idem pero solo con los caracteres especiales (&, ", ', <, >, ...)